

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf\_98c1f280-b9f\agent-aba95ab0315c2b7eb.jsonl`
- SHA-256: `eb458f2f4206293bb386b4cbeefc582257f0e2bfc885a04d252b54d9f2af6546`
- Source modified: `2026-06-14T06:34:24+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf\_98c1f280-b9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T06:32:15.325Z)

In the current badminton-highlight-indexer repo (cwd), audit ONE signal/component. Verify and return:

- present: the code exists.
- default_off: it does NOT run in the served/default path by default (e.g. cue_flags default False, default_segmenter is NOT 'fusion', the feature is not weighted by the shipped scorer unless explicitly enabled).
- no_special_casing: the signal enters the DECISION as a confidence-scored / feature COLUMN the learned scorer weights ? NOT a hard if/else branch hard-coded into the decision path.
- retained: NOTHING deletes or disables the code because the n=6 measured ?F1 was non-significant; the producer + feature math still exist and are re-measurable when the corpus grows.

Give evidence as concrete file:line refs with short quotes. List concerns. Be precise and skeptical; read the actual files.

SIGNAL: COURT BOUNDS (the catalyst-later one). Read fusion_features.py (point_in_polygon, person_in_court foot-point, in_court_counts, players_in_court, in_court_ratio), backend/config/models.py (FusionConfig.court_polygon / net_axis), and backend/eval/fusion_golden.py (how court_polygon is passed ? currently None). KEY QUESTION: is the court-polygon MATH present and does it ACTIVATE when polygons are later supplied (so it becomes a catalyst once Roora labels arrive), even though it degrades to "count everyone" with None today?

2. assistant (2026-06-14T06:32:18.697Z)

I'll audit the COURT BOUNDS signal. Let me start by pulling the repo and reading the key files.

3. user (2026-06-14T06:32:21.964Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-14T06:32:22.066Z)

backend\pipeline\detectors\fusion_features.py

5. user (2026-06-14T06:32:22.076Z)

backend\eval\fusion_golden.py

6. user (2026-06-14T06:32:22.086Z)

backend\config\models.py

7. assistant (2026-06-14T06:32:35.587Z)

The pull failed due to a config issue, not a real problem. Let me proceed read-only and check the git state, then read the files.

8. user (2026-06-14T06:32:37.365Z)

9eb06f4 docs(quality): log fusion P3+P4 measured ?F1 on the n=6 golden corpus
docs/i3-fusion-measured

9. user (2026-06-14T06:32:37.642Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of scale/translation-invariant
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12     - `per_frame` is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13     index (so it aligns directly with the shuttle trajectory's `frame` ? no clip re-
timing).
14     - A shuttle point has `frame`, `visible`, `x`, `y` (pixels) ? the WASB
trajectory shape.
15     - `court_polygon` is normalised 0-1 `(x, y), ...)` or `None` (None ? no mask,
every
16     detection counts ? the player-count-only mode).
17     - `net` is `(axis 'x'|'y', position 0-1 normalised)` or `None` (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except `mean_player_motion` (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
```

```

48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71         point meaningless, so a masked check returns False rather than testing a bogus
72         (0,0)."""
73         if court_polygon is None:
74             return True
75         if frame_width <= 0 or frame_height <= 0:
76             return False
77         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                       frame_height: float,
81                       court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82         """Per-frame count of in-court persons (one entry per sampled frame)."""
83         return [
84             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
85 court_polygon))
86             for _frame_idx, dets in sorted(per_frame.items())
87         ]
88
89     def _median(vals: Sequence[float]) -> float:
90         if not vals:
91             return 0.0
92         s = sorted(vals)
93         n = len(s)
94         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97     def players_in_court(counts: Sequence[int]) -> float:
98         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99         return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
104 flicker)."""
105         if not counts:
106             return 0.0
107         return sum(1 for c in counts if c >= min_players) / len(counts)

```

```

107
108
109 def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                        frame_width: float, frame_height: float) -> float:
111     """Mean per-track centre displacement between consecutive sampled frames, with x
112     normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113     not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114     if frame_width <= 0 or len(per_frame) < 2:
115         return 0.0
116     frames = sorted(per_frame.keys())
117     steps: List[float] = []
118     for a, b in zip(frames, frames[1:]):
119         prev = {tid: box for tid, box in per_frame[a]}
120         for tid, box in per_frame[b]:
121             if tid in prev:
122                 cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                 cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                 steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125     if not steps:
126         return 0.0
127     return sum(steps) / len(steps)
128
129
130 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                     frame_height: float, net: Optional[Net],
132                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137     0.0.
138     """
139     if net is None or not per_frame:
140         return 0.0
141     axis, pos = net
142     both = 0
143     for _frame_idx, dets in per_frame.items():
144         near, far = False, False
145         for _tid, box in dets:
146             if not person_in_court(box, frame_width, frame_height, court_polygon):
147                 continue
148             fx, fy = _foot_norm(box, frame_width, frame_height)
149             coord = fx if axis == "x" else fy
150             if coord < pos:
151                 near = True
152             else:
153                 far = True
154         if near and far:
155             both += 1
156     return both / len(per_frame)
157
158 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                    pad_frac: float) -> bool:
160     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161     width/height (the 'adjacent / in-hand' tolerance)?"""
162     x1, y1, x2, y2 = box
163     px = pad_frac * frame_width
164     py = pad_frac * frame_height
165     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,

```

```

169         frame_width: float, frame_height: float,
170         pad_frac: float = 0.03) -> float:
171     """Fraction of (visible shuttle frames that were also person-sampled) in which the
172     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
173     ~0**
174     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
175     """
176     if not per_frame or frame_width <= 0 or frame_height <= 0:
177         return 0.0
178     aligned = 0
179     colocated = 0
180     for p in shuttle_points or []:
181         if not getattr(p, "visible", False):
182             continue
183         dets = per_frame.get(int(p.frame))
184         if dets is None: # shuttle frame wasn't person-sampled ? can't judge
185             continue
186         aligned += 1
187         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
188             for _tid, box in dets):
189             colocated += 1
190     return colocated / aligned if aligned else 0.0
191
192 # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193 # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194 PERSON_FEATURE_NAMES = [
195     "players_in_court", "two_sided_motion", "mean_player_motion",
196     "in_court_ratio", "shuttle_player_colocation",
197 ]
198
199
200 def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
201 shuttle_points,
202                             frame_width: float, frame_height: float,
203                             court_polygon: Optional[Sequence[Tuple[float, float]]] =
204 None,
205                             net: Optional[Net] = None,
206                             min_players: int = 2) -> Dict[str, float]:
207     """Aggregate the per-window person features from per-frame detections + the in-
208     window
209     shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
210     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
211     return {
212         "players_in_court": players_in_court(counts),
213         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
214         court_polygon),
215         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
216         "in_court_ratio": in_court_ratio(counts, min_players),
217         "shuttle_player_colocation": shuttle_player_colocation(
218             per_frame, shuttle_points, frame_width, frame_height),
219     }

```

10. assistant (2026-06-14T06:32:43.513Z)

The fusion_features.py is fully read. Now let me read the config models and fusion_golden.py.

11. user (2026-06-14T06:32:44.429Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5

```

```

6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11    from typing import Any, List, Literal, Optional
12
13    from pydantic import BaseModel, Field, field_validator, model_validator
14
15    Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18    class ValidationRelevanceConfig(BaseModel):
19        court_green_lower: list[int] = Field(default=[35, 40, 40])
20        court_green_upper: list[int] = Field(default=[85, 255, 255])
21        court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24    class ValidationConfig(BaseModel):
25        min_resolution_width: int = 1280
26        min_resolution_height: int = 720
27        min_fps: float = 29.9
28        min_blur_threshold: float = 20.0
29        max_scene_cuts_per_minute: float = 0.5
30        relevance: ValidationRelevanceConfig =
31    Field(default_factory=ValidationRelevanceConfig)
32
33    class TrackNetConfig(BaseModel):
34        wsl_distro: str = "Ubuntu"
35        repo_dir: str = "~/models/TrackNetV4"
36        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37        conda_env: str = "TrackNetV4"
38        weights_path: str = ""
39        queue_length: int = 5
40        stage_in_wsl: bool = True
41        wsl_stage_dir: str = "~/clips"
42        timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43        velocity_threshold: float = 0.02
44        # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45        # decoded frame cache) are deleted automatically ? they are pure intermediates,
46        # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47        # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48        keep_frames: bool = False
49        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50        # 0 = disabled.
51        min_free_gb: float = 0.0
52
53
54    class WasbIndexConfig(BaseModel):
55        """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57        Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58        these knobs actually take effect ? previously this whole block was silently dropped
59        because nested config sections defaulted to extra='ignore'.
60        """
61        wsl_distro: str = "Ubuntu"
62        repo_dir: str = "~/models/WASB-SBDT"
63        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64        conda_env: str = "wasb"
65        weights_path: str = "~/models/WASB-
66    SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
67        sport: str = "badminton"
68        wsl_stage_dir: str = "~/clips"
69        timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)

```

```

69     velocity_threshold: float = 0.02
70     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73     keep_frames: bool = False
74     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75     min_free_gb: float = 0.0
76
77
78     class FusionConfig(BaseModel):
79         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
80
81         Every default reproduces control behaviour: the fusion segmenter persists the
82         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
83         true ? the person-cue features, but it does NOT gate the served handoff unless a
84         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
85         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
86         supplied ? off-court persons (spectators / adjacent courts) are excluded.
87         """
88         # Which trajectory substrate seeds the windows (shuttle stays primary).
89         substrate: Literal["wasb", "tracknet"] = "wasb"
90         # Windowing velocity threshold for the fusion family (the engine reads
91         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
92         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
93         velocity_threshold: float = 0.02
94         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
95         # enabled ? so the default path never imports torch / touches the GPU.
96         cue_flags: dict[str, bool] = Field(default_factory=dict)
97         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
98         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
99         min_crossings: int = Field(default=0, ge=0)
100         # Served Gate B: when the person cue is on, drop windows with median in-court
players
101         # < this. 0 = OFF.
102         min_players: int = Field(default=0, ge=0)
103         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
104         court_polygon: Optional[list[list[float]]] = None
105         # Net line for the person two-sided-motion split (person features only ? the shuttle
106         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
107         net_axis: Literal["x", "y"] = "y"
108         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
109
110
111     class IndexingConfig(BaseModel):
112         default_segmenter: str = "yolo_hybrid"
113         use_gpu: bool = True
114         motion_threshold: float = 0.08
115         min_segment_duration: float = 3.0
116         dead_time_merge_gap: float = 2.0
117         chunk_duration: float = 90.0
118         chunk_overlap: float = 10.0
119         detector_model: str = "fasterrcnn_resnet50_fpn"
120         detector_score_threshold: float = 0.5
121         yolo_velocity_threshold: float = 0.15
122         yolo_frame_skip: int = 3
123         yolo_tracker: str = "bytetrack.yaml"

```

```

124     yolo_prefetch_workers: int = 2
125     min_action_window_duration: float = 2.0
126     log_yolo_candidates: bool = True
127     store_yolo_candidates: bool = True
128     ai_handoff_padding: float = 2.0
129     ai_max_workers: int = 5
130     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
131     # chunks of a high-bitrate (4K) source blow past the provider upload cap
132     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
133     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
134     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
135     # Matches gemini_refine's --clip-scale-width default.
136     ai_chunk_scale_width: int = 1280
137     # Optional debug knob: trim every video to the first N seconds before processing.
138     debug_duration: Optional[float] = None
139
140     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
141     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
142     fusion: FusionConfig = Field(default_factory=FusionConfig)
143
144     @field_validator("default_segmenter")
145     @classmethod
146     def segmenter_must_be_registered(cls, v: str) -> str:
147         # Registry check happens at runtime in main/segmenter selection.
148         # Here we just allow any string for forward compatibility.
149         return v
150
151     @model_validator(mode="after")
152     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
153         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
154         # step makes `while t < duration: t += step` loop forever, growing memory before
155         # GPU work. Reject the bad config at load time instead.
156         if self.chunk_overlap >= self.chunk_duration:
157             raise ValueError(
158                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
159                 "
160                 f"({self.chunk_duration}); otherwise chunking never advances."
161             )
162         return self
163
164     class OutputConfig(BaseModel):
165         default_highlights_name: str = "highlights.mp4"
166         db_name: str = "sports_indexer.db"
167         # Directory where the DB, highlight reels, and processed clips are written.
168         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
169         output_dir: str = "output"
170
171     class WatermarkConfig(BaseModel):
172         """Branding overlay burned into each highlight clip during the per-clip re-encode
173         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
174         (under `stitching.watermark` in config.json) to turn it off. The text is fully
175         configurable. The concat stage stays stream-copy (-c copy)."""
176         enabled: bool = True
177         text: str = "Generated by Khelsutra.guru"
178         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
179         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
180         below
181         font: str = "Sans" # fontconfig family used when font_path is empty
182         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
183         fraction of frame height
184         opacity: float = Field(default=0.85, ge=0.0, le=1.0)

```



```

184         box: bool = True                # faint backing box behind the text for legibility
185         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
186         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
187
188
189     class StitchingConfig(BaseModel):
190         begin_padding: float = 0.5
191         end_padding: float = 0.5
192         use_cache: bool = False
193         min_shot_count: int = 1
194         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
195
196
197     class LoggingConfig(BaseModel):
198         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
199         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
200         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
201         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
202         # them to WARNING; set true to restore full chatter for request-flow debugging.
203         verbose_http: bool = False
204
205
206     class SecurityConfig(BaseModel):
207         # When set, /api/process video_path must resolve under this directory (hosted/tenant
208         # mode). Default None preserves the single-user local 'any local file' workflow.
209         allowed_video_root: Optional[str] = None
210
211         # --- Chunked upload (B2, PR-2) ---
212         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
213         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
214         # contain upload_dir or /api/process will reject the uploaded paths.
215         upload_dir: str = "output/uploads"
216         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
217         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
218         max_upload_mb: int = 4096
219         max_part_mb: int = 95
220         allowed_upload_extensions: List[str] = ["mp4", "mov"]
221
222
223     class AppConfig(BaseModel):
224         """Root configuration object.
225
226         All runtime configuration should be accessed via an instance of this class
227         (or the loader) rather than raw dict lookups.
228         """
229
230         sport: Sport = "badminton"
231         ai_provider: str = "gemini"
232         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
233         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
234         ai_model: str = "gemini-flash-latest"
235
236         validation: ValidationConfig = Field(default_factory=ValidationConfig)
237         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
238         output: OutputConfig = Field(default_factory=OutputConfig)
239         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
240         security: SecurityConfig = Field(default_factory=SecurityConfig)
241         logging: LoggingConfig = Field(default_factory=LoggingConfig)
242
243         # Allow extra keys during transition so existing config.json files do not break.
244         model_config = {
245             "extra": "allow",

```

```

246         "validate_assignment": True,
247     }
248
249     # Temporary migration shim for legacy dict-style access (config.get("section",
250     {}).get(...)).
251     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
252     # attribute access / the typed config. A nested *section* is returned as a plain
dict
253     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
254     # section model instead breaks it (a model has no .get), which bit every validator
on
255     # real runs. Scalar leaves are returned as-is.
256     def get(self, key: str, default: Any = None) -> Any:
257         _MISSING = object()
258         val: Any = _MISSING
259         if key in type(self).model_fields or hasattr(self, key):
260             val = getattr(self, key, _MISSING)
261         if val is _MISSING:
262             # Legacy convenience: find a leaf key inside any nested section. Sections
are
263             # discovered dynamically from the model fields, so a future top-level
section
264             # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
265             for fname in type(self).model_fields:
266                 section = getattr(self, fname, None)
267                 if isinstance(section, BaseModel) and hasattr(section, key):
268                     val = getattr(section, key)
269                     break
270             if val is _MISSING:
271                 return default
272             if isinstance(val, BaseModel):
273                 return val.model_dump(mode="json")
274             return val
275     def __getitem__(self, key: str) -> Any:
276         val = self.get(key)
277         if val is None and key not in type(self).model_fields:
278             # Re-raise like a real dict for unknown top-level
279             raise KeyError(key)
280         return val
281
282
283     # Convenience type alias for callers that want to annotate
284     Config = AppConfig
285

```

12. user (2026-06-14T06:32:44.585Z)

```

1     """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3     Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4     candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5     EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6     writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7     CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8     measured by the shipped experiment harness; person features enter only as feature
columns the
9     learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):

```

```

12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17     from __future__ import annotations
18
19     import argparse
20     import json
21     import os
22     import time
23     from typing import Any, Dict, List, Optional
24
25     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates
31
32     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
33     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
34     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
35     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
36
37
38     def _derive_video_path(spec: Dict[str, Any]) -> str:
39         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
40         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
convention)."""
41         if spec.get("video_path"):
42             return str(spec["video_path"])
43         labels = str(spec["labels"])
44         if labels.endswith(".rallies.csv"):
45             return labels[: -len(".rallies.csv")] + ".MP4"
46         raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
47
48
49     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
50                     frame_skip: int = 3, iou: float = 0.5,
51                     log_every_sec: float = 15.0) -> Dict[str, Any]:
52         """One golden video -> a feature record (shuttle + person features per candidate
window).
53
54         Returns `{name, fps, frame_width,
candidates: [{start, end, shuttle}, {person}, {label}], gts}`.
55         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
tests;
56         when None a real `PersonDetector` is built (loads torchvision on first use).
57         """
58         fps, fw = float(spec["fps"]), float(spec["frame_width"])
59         traj = str(spec["trajectory"])
60         points = TrackNetRunner.parse_trajectory_csv(traj)
61         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
62         gts = load_golden_gts(str(spec["labels"]))
63         labels = label_candidates(intervals, gts, iou)
64         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
65         # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
66         axis, net_px, _span = rally_gate.estimate_play_axis(points)

```

```

67     video_path = _derive_video_path(spec)
68
69     if detector is None:
70         from backend.pipeline.detectors.person_detector import PersonDetector
71         detector = PersonDetector({"indexing": {"use_gpu": True}})
72
73     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
74     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
75     n = len(intervals)
76     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
77     print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
78     t0 = time.monotonic()
79     state = {"last": t0}
80
81     def _progress(done: int, total: int) -> None:
82         now = time.monotonic()
83         if now - state["last"] >= log_every_sec or done >= total:
84             el = now - t0
85             rate = done / el if el > 0 else 0.0
86             eta = (total - done) / rate if rate > 0 else 0.0
87             pct = 100 * done // total if total else 0
88             print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
89                 f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
90             state["last"] = now
91
92     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
93                                                  progress_cb=_progress)
94
95     candidates: List[Dict[str, Any]] = []
96     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
97         det = per_window[i]
98         det_fw = det.get("frame_width") or fw
99         det_fh = det.get("frame_height") or 0.0
100         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
101         # normalisation fusion_features uses (x/width, y/height).
102         if axis == "x":
103             net_norm = net_px / det_fw if det_fw > 0 else 0.5
104         else:
105             net_norm = net_px / det_fh if det_fh > 0 else 0.5
106         sf, ef = win_frames[i]
107         shuttle_in = [p for p in points if sf <= p.frame <= ef]
108         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
109                                           court_polygon=None, net=(axis, net_norm))
110         candidates.append({
111             "start": float(s), "end": float(e),
112             "shuttle": {k: float(v) for k, v in zip(Shuttle_Feature_Names, xs)},
113             "person": {k: float(v) for k, v in person.items()},
114             "label": int(labels[i]),
115         })
116     return {
117         "name": spec["name"], "fps": fps, "frame_width": fw,
118         "candidates": candidates,
119         "gts": [[float(a), float(b)] for a, b in gts],
120     }
121
122
123 def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
124        fresh: bool = False, smallest_first: bool = False) -> List[str]:
125     with open(manifest_path, encoding="utf-8") as f:
126         specs = json.load(f)
127     if smallest_first:
128         # Process small videos first for quick feedback (the slow full match lands

```

```

last);
129         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
130         specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
131                                     if os.path.isfile(s.get("trajectory", ""))
else 0))
132         os.makedirs(out_dir, exist_ok=True)
133         written: List[str] = []
134         for vi, spec in enumerate(specs, 1):
135             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
136             # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
137             # run left off instead of re-detecting). --fresh forces a redo.
138             if not fresh and os.path.isfile(out):
139                 print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
140                       f"-> {out} (use --fresh to redo)", flush=True)
141                 written.append(out)
142                 continue
143             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
144             t0 = time.monotonic()
145             data = extract_video(spec, frame_skip=frame_skip, iou=iou)
146             n_pos = sum(c["label"] for c in data["candidates"])
147             with open(out, "w", encoding="utf-8") as f:
148                 json.dump(data, f, indent=2)
149             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
150                   f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
151                   f"-> {out}", flush=True)
152             written.append(out)
153         return written
154
155
156     def main() -> None:
157         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
158         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
159         ap.add_argument("--out-dir", default="output")
160         ap.add_argument("--frame-skip", type=int, default=3)
161         ap.add_argument("--iou", type=float, default=0.5)
162         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
163         ap.add_argument("--smallest-first", action="store_true",
164                           help="process smaller videos first for quick feedback (LOVO is
order-independent)")
165         args = ap.parse_args()
166         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
167             smallest_first=args.smallest_first)
168
169
170     if __name__ == "__main__":
171         main()
172

```

13. assistant (2026-06-14T06:32:48.864Z)

Now let me verify how the court_polygon flows into the production/served fusion segmenter (not just the eval path), and confirm the config default is None. Let me find the fusion segmenter that wires `compute\_person\_features`.

14. user (2026-06-14T06:32:50.091Z)

docs\CODE_MAP.md:114:[Omitted long matching line]

```

docs\CODE_MAP.md:130:[Omitted long matching line]
docs\NEXT_STEPS.md:72: optional `fusion.court_polygon`. Adversarially reviewed (31 agents,
19 confirmed; NO-REGRESSION
backend\config\models.py:85: is optional: ``court_polygon=None`` ? Gate B counts every
detection (player-count-only);
backend\config\models.py:104: court_polygon: Optional[list[list[float]]] = None
docs\MULTI_SIGNAL_FUSION_PLAN.md:278: superset** (offline substrate intact). Court mask =
optional `fusion.court_polygon` (None ? count-only).
backend\eval\fusion_golden.py:108: person = ff.compute_person_features(det["frames"],
shuttle_in, det_fw, det_fh,
backend\eval\fusion_golden.py:109:
court_polygon=None, net=(axis, net_norm))
tests\test_config.py:51: assert f.court_polygon is None # no mask
tests\test_config.py:61: "cue_flags": {"person": True}, "court_polygon": [[0.1,
0.1], [0.9, 0.1], [0.5, 0.9]],
tests\test_config.py:68: assert len(f.court_polygon) == 3
tests\test_fusion_features.py:36:#
----- person_in_court
tests\test_fusion_features.py:38:def test_person_in_court_uses_foot_point_and_polygon():
tests\test_fusion_features.py:42: assert ff.person_in_court(in_box, 100, 100, LEFT_HALF) is
True
tests\test_fusion_features.py:43: assert ff.person_in_court(out_box, 100, 100, LEFT_HALF) is
False
tests\test_fusion_features.py:45: assert ff.person_in_court(out_box, 100, 100, None) is True
tests\test_fusion_features.py:48: assert ff.person_in_court(in_box, 0, 0, LEFT_HALF) is False
tests\test_fusion_features.py:49: assert ff.person_in_court(in_box, 0, 0, None) is True
tests\test_fusion_features.py:54:def test_in_court_counts_and_players_and_ratio():
tests\test_fusion_features.py:59: counts = ff.in_court_counts(per_frame, 100, 100, LEFT_HALF)
tests\test_fusion_features.py:65: assert ff.in_court_counts(per_frame, 100, 100, None) == [2,
1, 0, 2]
tests\test_fusion_features.py:94:def test_two_sided_motion_respects_court_polygon():
tests\test_fusion_features.py:131:def test_compute_person_features_keys():
tests\test_fusion_features.py:136: feats = ff.compute_person_features(per_frame, shuttle,
100, 100,
tests\test_fusion_features.py:137: court_polygon=None,
net=("y", 0.5))
tests\test_fusion_features.py:143:def test_compute_person_features_empty():
tests\test_fusion_features.py:144: feats = ff.compute_person_features({}, [], 100, 100)
backend\pipeline\detectors\fusion_features.py:15:- ``court_polygon`` is normalised 0-1 ``[(x,
y), ...]`` or ``None`` (None ? no mask, every
backend\pipeline\detectors\fusion_features.py:67:def person_in_court(box: BBox, frame_width:
float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:68: court_polygon:
Optional[Sequence[Tuple[float, float]]]) -> bool:
backend\pipeline\detectors\fusion_features.py:72: if court_polygon is None:
backend\pipeline\detectors\fusion_features.py:76: return point_in_polygon(_foot_norm(box,
frame_width, frame_height), court_polygon)
backend\pipeline\detectors\fusion_features.py:79:def in_court_counts(per_frame: Dict[int,
List[Tuple[int, BBox]]], frame_width: float,
backend\pipeline\detectors\fusion_features.py:81: court_polygon:
Optional[Sequence[Tuple[float, float]]]) -> List[int]:
backend\pipeline\detectors\fusion_features.py:84: sum(1 for _tid, box in dets if
person_in_court(box, frame_width, frame_height, court_polygon))
backend\pipeline\detectors\fusion_features.py:132: court_polygon:
Optional[Sequence[Tuple[float, float]]]) -> float:
backend\pipeline\detectors\fusion_features.py:145: if not person_in_court(box,
frame_width, frame_height, court_polygon):
backend\pipeline\detectors\fusion_features.py:200:def compute_person_features(per_frame:
Dict[int, List[Tuple[int, BBox]]], shuttle_points,
backend\pipeline\detectors\fusion_features.py:202: court_polygon:
Optional[Sequence[Tuple[float, float]]] = None,
backend\pipeline\detectors\fusion_features.py:207: counts = in_court_counts(per_frame,
frame_width, frame_height, court_polygon)
backend\pipeline\detectors\fusion_features.py:210: "two_sided_motion":
two_sided_motion(per_frame, frame_width, frame_height, net, court_polygon),

```

```

backend\pipeline\segmenters\fusion_hybrid.py:126:         poly = fcfg.get("court_polygon")
backend\pipeline\segmenters\fusion_hybrid.py:130:         return
ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
backend\pipeline\segmenters\fusion_hybrid.py:131:
court_polygon=polygon, net=net)

```

15. user (2026-06-14T06:32:50.603Z)

```

config.json:26:         "default_segmenter": "gemini",
README.md:74:This outputs a validation summary and indexes all rallies directly into the local
SQLite database, using whichever segmenter is set as `default_segmenter` in `config.json`. **The
shipped `config.json` sets `gemini`** (needs `GEMINI_API_KEY`; without it the run silently
produces 0 segments and the observability report flags `silent_provider_noop`). Set it to
`yolo_hybrid`/`wasb_hybrid`/`motion` for the local paths.
README.md:142:         "default_segmenter": "gemini",
README.md:180:*   `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
README.md:242:   *Or set `default_segmenter`: "yolo_hybrid" (or `gemini`/`motion`) under
`indexing` in `config.json`.*
README.md:246:[Omitted long matching line]
README.md:264:         "default_segmenter": "gemini",
run_process_and_stitch.py:74:         seg_name = cfg.indexing.default_segmenter
backend\main.py:148:     default_seg = getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
backend\main.py:712:     seg_name = args.segmenter or getattr(getattr(global_config,
"indexing", None), "default_segmenter", "motion")
tests\test_config.py:23:     assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:78:         "default_segmenter": "gemini",
tests\test_config.py:89:     assert cfg.indexing.default_segmenter == "gemini"
tests\test_config.py:102:         "indexing": {"default_segmenter": "yolo_hybrid"},
tests\test_config.py:110:     assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:138:     "indexing": {"default_segmenter": "gemini", "wasb":
{"velocity_threshold": 0.02}},
tests\test_config.py:154:     assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:198:     assert cfg.get("indexing", {}).get("default_segmenter") ==
"yolo_hybrid"
backend\config\models.py:79:     """Config for the multi-signal `fusion_hybrid` segmenter
(MULTI_SIGNAL_FUSION_PLAN P2).
backend\config\models.py:112:     default_segmenter: str = "yolo_hybrid"
backend\config\models.py:144:     @field_validator("default_segmenter")
backend\reporting\spec.yaml:133:     The segmenter that ran differs from config.json's
default_segmenter, and no
backend\reporting\harvest.py:216:     config_default = (config.get("indexing") or
{}).get("default_segmenter")
tests\test_fusion_hybrid.py:12:from backend.pipeline.segmenters.fusion_hybrid import
FusionSegmenter
tests\test_fusion_hybrid.py:143:     with
patch("backend.pipeline.segmenters.fusion_hybrid.WasbRunner") as W, \
tests\test_fusion_hybrid.py:144:
patch("backend.pipeline.segmenters.fusion_hybrid.WasbConfig") as WC, \
tests\test_fusion_hybrid.py:145:
patch("backend.pipeline.segmenters.fusion_hybrid.TrackNetRunner") as T, \
tests\test_fusion_hybrid.py:146:
patch("backend.pipeline.segmenters.fusion_hybrid.TrackNetConfig") as TC:
docs\CODE_MAP.md:31: -> seg_name = req.segmenter or global_config.indexing.default_segmenter
(fallback 'motion')
docs\CODE_MAP.md:89: - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
docs\CODE_MAP.md:99:[Omitted long matching line]
docs\CODE_MAP.md:105:- `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
docs\CODE_MAP.md:112:- `@backend/pipeline/segmenters/__init__.py` ? imports all 6
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid,fusion_hybrid`) = the de-facto
registration manifest; re-exports them + `segmenter_registry` via `__all__`.

```

```

docs\CODE_MAP.md:113:[Omitted long matching line]
docs\CODE_MAP.md:114:[Omitted long matching line]
docs\CODE_MAP.md:131:[Omitted long matching line]
docs\CODE_MAP.md:186:[Omitted long matching line]
docs\CODE_MAP.md:250:- **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid`; `config.json` may override at runtime (ships
`gemini`). run*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.
docs\CODE_MAP.md:276:| Change a config default / add a config field |
`@backend/config/models.py` (single source of truth) -> regenerate via
`setup.py`/`save_default_config`; ? config.json may override (default_segmenter) |
docs\CODE_MAP.md:314:| `test_fusion_hybrid.py` |
`pipeline/segmenters/fusion_hybrid.FusionSegmenter` (net_crossings persisted; superset; Gate A &
Gate B handoff gating; person-cue features; `_build_runner` substrate select) |
docs\EXPERIMENT_HARNESS.md:106:- `backend/config/models.py::IndexingConfig` ?
`default_segmenter` (:65, default `"yolo_hybrid"`), `detector_model` (:72),
`detector_score_threshold` (:73), `yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75),
`min_segment_duration` (:68), `dead_time_merge_gap` (:69), `motion_threshold` (:67). **No
`min_crossings` knob yet** (see fusion P2).
docs\EXPERIMENT_HARNESS.md:123:JSON-serializable; selected via the existing `segmenter_registry`
+ `IndexingConfig.default_segmenter`.
docs\EXPERIMENT_HARNESS.md:155:pinned ? **rollback = flip `default_segmenter`/variant config,
never a code revert.**
docs\EXPERIMENT_HARNESS.md:185:3. **Per-cue enable flags** in `IndexingConfig` (default
`False`); `default_segmenter` already exists.
docs\TRACKNET_WSL_SETUP.md:71:Then select it as the segmenter (e.g. set
`indexing.default_segmenter` to
docs\ANALYZERS_AND_RECONCILERS.md:184:The FusionSegmenter hook (`fusion_hybrid.py:70`) already
turns "compute fixed features" into per-window
docs\ANALYZERS_AND_RECONCILERS.md:417: (`fusion_hybrid.py:70`) is the exact injection point
§3.3 generalizes; `net_crossings` and the person
docs\MULTI_SIGNAL_FUSION_PLAN.md:28:| `yolo_hybrid` (the current `default_segmenter`) | person
velocity (torchvision + ByteTrack) | `backend/pipeline/segmenters/yolo_hybrid.py` |
docs\MULTI_SIGNAL_FUSION_PLAN.md:270: Shipped
`backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (registry `fusion_hybrid`,
docs\MULTI_SIGNAL_FUSION_PLAN.md:271: **default-OFF** ? `default_segmenter` unchanged),
subclassing the trajectory engine via two identity
docs\NEXT_STEPS.md:67: 4. ? **DONE (2026-06-14) ? Fusion P2 ? the FusionSegmenter:**
`fusion_hybrid.py::FusionSegmenter`
docs\NEXT_STEPS.md:68: (registry `fusion_hybrid`, **default-OFF**) seeds windows from the
shuttle substrate, persists
backend\pipeline\segmenters\__init__.py:7:from backend.pipeline.segmenters.fusion_hybrid import
FusionSegmenter
backend\pipeline\segmenters\fusion_hybrid.py:24:and hands off identically to `wasb_hybrid`.
Registered as `fusion_hybrid`, NOT the default
backend\pipeline\segmenters\fusion_hybrid.py:56: return "fusion_hybrid"
docs\QUALITY_ITERATIONS.md:139:**Shipping shape.** A NEW `fusion` segmenter, **default OFF**
(`default_segmenter` unchanged); Gate A
docs\QUALITY_ITERATIONS.md:146:**Status (2026-06-14): SHIPPED, default-OFF.** `FusionSegmenter`
(`backend/pipeline/segmenters/fusion_hybrid.py`,
docs\QUALITY_ITERATIONS.md:147:registry key `fusion_hybrid`) + the pure feature math
(`backend/pipeline/detectors/fusion_features.py`) + two
tests\test_reporting_harvest.py:46: config={"ai_provider": "gemini", "ai_model": "gemini-
flash", "indexing": {"default_segmenter": "wasb_hybrid"}},
tests\test_reporting_harvest.py:122: config={"ai_provider": "gemini", "indexing":
{"default_segmenter": "gemini"}},
tests\test_reporting_harvest.py:137: config={"indexing": {"default_segmenter":
"motion"}})
tests\test_reporting_harvest.py:146: cfg = {"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}}
tests\test_reporting_harvest.py:189: config={"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}},

```


The court_polygon flows into the production segmenter at `fusion\_hybrid.py:126-131`. Let me read that segmenter's relevant section to verify how polygon is passed and whether the person cue is gated default-OFF, and how the features enter the decision (scored column vs hard if/else).

17. user (2026-06-14T06:32:56.008Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13     torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14     model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15     via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16     - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
experiment
20     harness can re-score any variant.
21
22     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
```

```

48         self._person: Optional[Any] = None
49         # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50         # over the whole trajectory once per candidate window (it depends only on
`points`).
51         self._axis_cache_key: Optional[int] = None
52         self._axis_cache: Optional[tuple] = None
53
54     @property
55     def name(self) -> str:
56         return "fusion_hybrid"
57
58     @property
59     def description(self) -> str:
60         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                 "AI provider sets semantic boundaries.")
63
64     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66         if substrate == "tracknet":
67             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68             return TrackNetRunner(cfg), {
69                 "weights_path": cfg.weights_path,
70                 "queue_length": cfg.queue_length,
71                 "fusion_substrate": "tracknet",
72             }
73         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74         return WasbRunner(wcfg, {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"})
75
76     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                frame_width: float, video_path: str,
78                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                              min_crossings=0,
estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92     def _axis_estimate(self, points: List[Any]) -> tuple:
93         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94         computed once per video, not once per candidate window (it depends only on
points)."""
95         key = id(points)
96         if self._axis_cache_key != key or self._axis_cache is None:
97             self._axis_cache_key = key
98             self._axis_cache = rally_gate.estimate_play_axis(points)

```

```

99         est = self._axis_cache
100         assert est is not None
101         return est
102
103     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                         frame_width: float, video_path: str,
105                         fcfg: Dict[str, Any]) -> Dict[str, float]:
106         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107         frame range and compute the scale/translation-invariant person features. Lazily
builds
108         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110         from backend.pipeline.detectors.person_detector import PersonDetector
111         if self._person is None:
112             self._person = PersonDetector(self.config)
113
114         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119         fw = det["frame_width"] or frame_width
120         fh = det["frame_height"]
121         # If we can't trust the frame dims, every normalised feature would silently
collapse
122         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123         if fw <= 0 or fh <= 0:
124             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126         poly = fcfg.get("court_polygon")
127         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                         court_polygon=polygon, net=net)
132
133     def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                           window_stats: List[Dict[str, Any]],
135                           idx_cfg: Dict[str, Any]) -> List[int]:
136         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137         (0) keep every window ? identical to the substrate. Persisted candidates are
138         untouched (full superset)."""
139         fcfg = idx_cfg.get("fusion", {})
140         min_crossings = int(fcfg.get("min_crossings", 0))
141         min_players = int(fcfg.get("min_players", 0))
142         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144         keep: List[int] = []
145         for i, st in enumerate(window_stats):
146             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                 continue
148             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                 continue
150             keep.append(i)
151         return keep
152
153
154     segmenter_registry.register(FusionSegmenter)

```

18. assistant (2026-06-14T06:33:02.984Z)

Now I have the full picture of the production wiring. Let me check the `\_select\_for\_handoff` and how the learned scorer weights these features (to verify "no special-casing" ? features enter as scored columns, not hard if/else in the decision path). Let me look at the distill_local FEATURE_NAMES and fusion_compare to understand how person features enter the learned scorer.

19. user (2026-06-14T06:33:04.518Z)

```
fusion_audio.py
fusion_compare.py
fusion_golden.py
```

20. user (2026-06-14T06:33:05.847Z)

```
backend\eval\distill_local.py:40:FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity",
"active_ratio", "net_crossings"]
backend\eval\distill_local.py:116:         model = LogisticRegression().fit(X, y,
FEATURE_NAMES)
backend\eval\distill_local.py:135:     final = LogisticRegression().fit(Xall, yall,
FEATURE_NAMES)
backend\eval\distill_local.py:137:     weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for
w in final.w]))
backend\eval\experiment.py:95:         whose persisted ``players_in_court`` is >= this. 0 = off
(no-op until the person
backend\eval\experiment.py:208:         col = _feature_column(vc, "players_in_court")
backend\eval\fusion_audio.py:23:from backend.eval.fusion_golden import PERSON_FEATURE_NAMES,
SHUTTLE_FEATURE_NAMES, _derive_video_path
backend\eval\fusion_audio.py:64:         for fn in SHUTTLE_FEATURE_NAMES:
backend\eval\fusion_audio.py:66:         for fn in PERSON_FEATURE_NAMES:
backend\eval\fusion_audio.py:68:         for fn in af.AUDIO_FEATURE_NAMES:
backend\eval\fusion_audio.py:81:         feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES), threshold=threshold)
backend\eval\fusion_audio.py:84:         feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES) + list(af.AUDIO_FEATURE_NAMES),
backend\eval\fusion_compare.py:22:from backend.eval.fusion_golden import PERSON_FEATURE_NAMES,
SHUTTLE_FEATURE_NAMES
backend\eval\fusion_compare.py:45:         for fn in SHUTTLE_FEATURE_NAMES:
backend\eval\fusion_compare.py:47:         for fn in PERSON_FEATURE_NAMES:
backend\eval\fusion_compare.py:57:     shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
backend\eval\fusion_compare.py:60:
feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
backend\eval\fusion_golden.py:33:SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
backend\eval\fusion_golden.py:34:# The 5 person-cue features
(fusion_features.PERSON_FEATURE_NAMES).
backend\eval\fusion_golden.py:35:PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
backend\eval\fusion_golden.py:109:
court_polygon=None, net=(axis, net_norm))
backend\eval\fusion_golden.py:112:         "shuttle": {k: float(v) for k, v in
zip(SHUTTLE_FEATURE_NAMES, xs)},
backend\eval\training.py:28:YOLO_FEATURE_NAMES = [
```

21. assistant (2026-06-14T06:33:10.350Z)

Let me read the fusion_compare.py to confirm the person features (including the court-dependent `players\_in\_court`/`in\_court\_ratio`) enter the learned LogisticRegression as feature columns, and confirm the n=6 non-significant result didn't trigger deletion. Let me also check the QUALITY_ITERATIONS doc for the measured ?F1 and retention statement.

22. user (2026-06-14T06:33:10.967Z)

```

1      """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3      The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4      and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5      no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6      designed seam): the person features are extra columns the learned LogisticRegression
weights,
7      so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9      Run (no GPU):
10     python -m backend.eval.fusion_compare --features-dir output --record
11     """
12     from __future__ import annotations
13
14     import argparse
15     import glob
16     import json
17     import os
18     from typing import Any, Dict, List
19
20     from backend.eval import experiment
21     from backend.eval.eval_stats import paired_delta_ci
22     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31         vids = [p["video"] for p in res["variant_a"]["per_video"]]
32         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35     def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36         """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37         feature columns + golden gts). The harness derives labels from gts itself."""
38         videos: List[experiment.VideoCandidates] = []
39         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40             with open(path, encoding="utf-8") as f:
41                 d = json.load(f)
42                 cands = d["candidates"]
43                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                 features: Dict[str, List[float]] = {}
45                 for fn in SHUTTLE_FEATURE_NAMES:
46                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                 for fn in PERSON_FEATURE_NAMES:
48                     features[fn] = [float(c["person"][fn]) for c in cands]
49                 gts = [(float(a), float(b)) for a, b in d["gts"]]
50                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                     features=features, gts=gts))
52         return videos
53
54
55     def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                 threshold: float = 0.5) -> Dict[str, Any]:
57         shuttle_only = experiment.Variant(name="shuttle_only",

```

```

feature_names=list(SHUTTLE_FEATURE_NAMES),
58                                     threshold=threshold)
59     fusion = experiment.Variant(name="shuttle_person_fusion",
60                                 feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
61                                     threshold=threshold)
62     return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
63
64
65     def format_result(res: Dict[str, Any], iou: float) -> str:
66         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
67         ds = delta_stats(res)
68         st = ds["sign_test"]
69         lines = [
70             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
71             f"    shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
72             f"    +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
73             f"    ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
74             f"    paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
75             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
76             f"| CI clears 0: {ds['ci_excludes_zero']}",
77             f"    per-video ?F1:",
78         ]
79         for pv in res["per_video_delta"]:
80             lines.append(f"        {pv['video']}<26s} ?F1={pv['f1']:+.3f}")
81         return "\n".join(lines)
82
83
84     def main() -> None:
85         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
86         ap.add_argument("--features-dir", default="output")
87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

23. assistant (2026-06-14T06:33:16.139Z)

The court-dependent person features (`players\_in\_court`, `in\_court\_ratio`, `two\_sided\_motion`) enter the learned LogisticRegression as feature columns. Now let me verify the "retained despite non-significant n=6 ?F1" claim by reading the QUALITY_ITERATIONS measured-result section and confirming nothing deletes/disables the code.

24. user (2026-06-14T06:33:18.860Z)

d7b1f51 feat(eval): honest ?F1 ? bootstrap CI + paired sign test on the fusion lift (#127)
39e961b feat(eval): fusion P3 measurement ? golden person-feature extractor + LOVO compare (#121)
cbf2236 feat(segmenters): FusionSegmenter ? shuttle + net-crossing + person-cue fusion (P2, default-OFF) (#117)

25. user (2026-06-14T06:33:20.158Z)

30:| Gen-0 owned head (LOVO, n=6) | 0.744 | 0.561 |
43:### Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio = promising (not promotable) *(2026-06-14)*
46:test; never a bare mean at n=6)**:
48:| Variant (honest LOVO, IoU ? 0.5) | F1 | ?F1 vs prior | 95% CI | sign test | CI clears 0 |
54:**Read honestly:** neither cue clears the promotion bar (both CIs span 0, both sign tests n.s.) at n=6 ? **both stay
57:(largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above** shuttle-only (0.307) ? but n=6 can't
64:(?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg 1.68?1.01 ? the
78:(Platt/isotonic, C2). Validated on the n=6 golden set: **mean F1 0.307 ? 0.423 (+0.116)**, testlarge
85:### First end-to-end A/B on the golden corpus (n=6) ? **ARCHIVED** *(2026-06-14)*
87:**Gate-A (net-crossings ? 2) is a measured win** ? ?F1 +0.074, 6/6 matches, sign-test **p = 0.031**, CI
184:- **`2026-06-multivideo-lovo/`** ? the **n=6 corpus + honest LOVO** ; established the heuristic is
187: ~+0.10** (n=6: learned 0.583 vs heuristic 0.480), with **AUC 0.83?0.92 decoupled from contrast**. This

26. assistant (2026-06-14T06:33:23.237Z)

Let me read the key retention/court-polygon catalyst section of QUALITY_ITERATIONS.md and the NEXT_STEPS.md court-polygon mention to confirm the "catalyst-later" framing and retention.

27. user (2026-06-14T06:33:24.197Z)

43 ### Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio = promising (not promotable) *(2026-06-14)*
44 The GPU person-feature extraction finished all 6 golden videos (`fusion\_golden.py` ? `output/\*\_golden\_features.json`, the
45 replayable substrate); the CPU LOVO drivers then measured the lift with **honest stats (C1 ? bootstrap 95% CI + paired sign
46 test; never a bare mean at n=6)**:
47
48 | Variant (honest LOVO, IoU ? 0.5) | F1 | ?F1 vs prior | 95% CI | sign test | CI clears 0 |
49 | ---|---|---|---|---|---|
50 | shuttle-only | 0.307 | ? | ? | ? | ? |
51 | + person (P3) | 0.286 | **?0.021** (?6.7%) | [?0.165, +0.162] | 2W/4L (p=0.688) | **No** |
52 | + person + audio (P4 incr.) | 0.366 | **+0.079** (+27.7%) | [?0.054, +0.203] | 4W/2L (p=0.688) | **No** |
53
54 **Read honestly:** neither cue clears the promotion bar (both CIs span 0, both sign tests n.s.) at n=6 ? **both stay
55 default-OFF** (promote-only-on-lift). **Person** adds noise (no court polygons ? `players\_in\_court` counts spectators);
56 per-video it's a wash (testlarge +0.40, gbaaddy ?0.22). **Audio** is the more promising lead ? +0.079 / +27.7%, wins 4/6
57 (largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above** shuttle-only (0.307) ? but n=6 can't
58 confirm it. **Next:** more matches (raise N until the sign test can resolve ~+0.08), and

```

measure audio on shuttle-only
59     *directly* (without the person handicap). Recorded in
`eval_baselines/experiment_leaderboard.json`. **Negative/uncertain
60     results are kept on purpose ? this is the honest ablation backbone for the paper aim.**
61
62     ### Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125
corroboration *(2026-06-14)*
63     **Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness).** The
measured win
64     (?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg
1.68?1.01 ? the
65     [first-A/B archive](archives/quality-iterations/2026-06-first-ab-experiment/README.md))
clears the honest
66     bar. **Recommended config:** `indexing.fusion.min_crossings = 2` on the fusion path (the
eval-only Gate A
67     folded into production, MULTI_SIGNAL_FUSION_PLAN $3.1). ? The offline win is on the
recall-oriented *proposal*

```

28. user (2026-06-14T06:33:25.126Z)

```

62     the EXISTING `calibration.py`, `metrics.py`, `classifier.py`, `training.py`,
`regression.gt_hash`, and
63     `database.query_candidate_segments` (no rebuilt scoring). Shipped `Variant` (+
pinned `CONTROL`),
64     `compare` (labeled A/B, LOVO-honest for learned variants), `compare_unlabeled`
(**SxS on new
65     unlabeled footage** ? agreed/A-only/B-only), `record_experiment` (leaderboard
JSON), and the DB
66     bridge; 19 tests incl. the no-regression invariant. Suite 421?440 green.
67     4. ? **DONE (2026-06-14) ? Fusion P2 ? the FusionSegmenter:**
`fusion_hybrid.py::FusionSegmenter`
68     (registry `fusion_hybrid`, **default-OFF**) seeds windows from the shuttle
substrate, persists
69     `net_crossings` (Gate A) + optional person features (`fusion_features.py`, pure-
tested) via two
70     identity hooks on the trajectory engine, with optional served Gate A/B
(`fusion.min_crossings` /
71     `min_players`, default 0) that gates the handoff while persisting the full
superset. Court mask =
72     optional `fusion.court_polygon`. Adversarially reviewed (31 agents, 19 confirmed;
NO-REGRESSION
73     verified); suite 441?467 green. Next: **P3** learned fusion (person features ?
harness `compare`
74     LOVO lift on the 6 golden videos; the eager-person-compute optimisation lands
here), then **P4**
75     audio cue via `backend/pipeline/audio/hit_detector.py`.
76     - **Discipline (from EXPERIMENT_HARNESS.md):** every new cue lands **flag-gated, default
OFF**; promote
77     to default **only on measured LOVO lift** through the `run_regression.py` gate (exit
0/1/3); pinned
78     control variant ? **rollback = config flip, never `git revert`**. The harness is ~80%
existing code.
79     - ? Dev container has no GPU/torch/cv2 ? Gate-A wiring, the person-cue extraction logic,
and
80     `experiment.py` are pure-logic/testable here; real-video confirmation is owner-side
(GPU/WSL).
81

```

29. assistant (2026-06-14T06:33:28.874Z)

I now have all the evidence I need. Let me do a final check on the test for
`compute\_person\_features` court-polygon activation to confirm the math is exercised and
activates when a polygon is supplied (the catalyst proof).

30. user (2026-06-14T06:33:29.811Z)

```
36      # ----- person_in_court
37
38      def test_person_in_court_uses_foot_point_and_polygon():
39          # box foot = ((x1+x2)/2/W, y2/H). W=H=100.
40          in_box = (10, 10, 30, 30)    # foot (0.2, 0.3) ? left half
41          out_box = (60, 10, 80, 30)   # foot (0.7, 0.3) ? right half
42          assert ff.person_in_court(in_box, 100, 100, LEFT_HALF) is True
43          assert ff.person_in_court(out_box, 100, 100, LEFT_HALF) is False
44          # None polygon => everyone counts
45          assert ff.person_in_court(out_box, 100, 100, None) is True
46          # Unusable frame dims => foot point is meaningless; a masked check returns False
47          # contract, not whatever the (0,0) ray-cast happens to give). No mask still counts
48          everyone.
49          assert ff.person_in_court(in_box, 0, 0, LEFT_HALF) is False
50          assert ff.person_in_court(in_box, 0, 0, None) is True
51
52      # ----- counts / ratios
53
54      def test_in_court_counts_and_players_and_ratio():
55          near = (10, 10, 30, 30)      # foot (0.2,0.3) in LEFT_HALF
56          far = (60, 10, 80, 30)       # foot (0.7,0.3) NOT in LEFT_HALF
57          per_frame = {0: [(1, near), (2, far)], 1: [(1, near)], 2: [], 3: [(1, near), (2,
58          near)]}
59          # With LEFT_HALF mask: counts = [1, 1, 0, 2]
60          counts = ff.in_court_counts(per_frame, 100, 100, LEFT_HALF)
61          assert counts == [1, 1, 0, 2]
62          assert ff.players_in_court(counts) == 1.0          # median of [0,1,1,2]
63          assert ff.in_court_ratio(counts, min_players=2) == pytest.approx(0.25)
64          assert ff.in_court_ratio(counts, min_players=1) == pytest.approx(0.75)
65          # No mask: far counts too -> [2,1,0,2]
66          assert ff.in_court_counts(per_frame, 100, 100, None) == [2, 1, 0, 2]
67          assert ff.players_in_court([]) == 0.0
68          assert ff.in_court_ratio([]) == 0.0
69
70      # ----- mean_player_motion
71
72      def test_mean_player_motion():
73          # track 1 moves 10px in x between frames; W=H=100 -> normalized disp 0.10.
74          per_frame = {0: [(1, (0, 0, 10, 10))], 1: [(1, (10, 0, 20, 10))]}
75          assert ff.mean_player_motion(per_frame, 100, 100) == pytest.approx(0.10)
76          # single frame -> 0; no shared track id -> 0
77          assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))]}, 100, 100) == 0.0
78          assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))], 1: [(2, (10, 0, 20, 10))]},
79          100, 100) == 0.0
80
81      # ----- two_sided_motion
82
83      def test_two_sided_motion():
84          net = ("y", 0.5)
85          near = (10, 10, 30, 40)      # foot y = 0.4 < 0.5 (near side)
86          far = (10, 60, 30, 90)       # foot y = 0.9 >= 0.5 (far side)
87          both = {0: [(1, near), (2, far)], 1: [(1, near)]}    # frame 0 two-sided, frame 1
88          one-sided
89          assert ff.two_sided_motion(both, 100, 100, net, None) == pytest.approx(0.5)
90          one_sided = {0: [(1, near)], 1: [(2, near)]}
91          assert ff.two_sided_motion(one_sided, 100, 100, net, None) == 0.0
92          assert ff.two_sided_motion(both, 100, 100, None, None) == 0.0    # no net -> 0
93
```

```

94     def test_two_sided_motion_respects_court_polygon():
95         net = ("y", 0.5)
96         near = (10, 10, 30, 40)          # foot (0.2, 0.4) ? near side, inside LEFT_HALF
97         far_out = (60, 60, 80, 90)      # foot (0.7, 0.9) ? far side BUT outside LEFT_HALF
98         per_frame = {0: [(1, near), (2, far_out)]}
99         assert ff.two_sided_motion(per_frame, 100, 100, net, None) == 1.0          # both sides
present
100         # the polygon excludes the far-side person -> only the near side counts -> not two-
sided
101         assert ff.two_sided_motion(per_frame, 100, 100, net, LEFT_HALF) == 0.0
102
103
104     def test_shuttle_player_colocation_pad_halo():
105         per_frame = {0: [(1, (40, 40, 60, 60))]}      # raw box x in [40,60]; default pad
0.03*100=3px halo

```

31. assistant (2026-06-14T06:33:35.405Z)

I have complete evidence. The court-polygon math is present, tested with a real polygon (`LEFT\_HALF`), proven to change counts/ratios/two-sided when supplied vs `None`, and the production segmenter (`fusion\_hybrid.py:126-131`) passes `fcfg.get("court\_polygon")` through ? so it activates the moment a polygon is configured. The default is `None` everywhere served (config default, golden eval passes `None` explicitly). The features enter the learned LogisticRegression as columns, and nothing was deleted after the n=6 non-significant result. Let me record my findings.

32. user (2026-06-14T06:34:10.041Z)

Structured output provided successfully

33. assistant (2026-06-14T06:34:24.152Z)

Audit complete. The COURT BOUNDS signal passes all four checks.

Summary of findings:

- present: yes. The polygon math is fully implemented and pure (no torch/cv2) in `backend/pipeline/detectors/fusion\_features.py` ? `point\_in\_polygon` (ray-casting, :30-48), `\_foot\_norm` (:51-57), `person\_in\_court` (:67-76), `in\_court\_counts` (:79-86), `players\_in\_court` (:97-99), `in\_court\_ratio` (:102-106).
- default_off: yes, on three layers. Config default `court\_polygon=None` and empty `cue\_flags` (`backend/config/models.py:96,104`); the served segmenter is `yolo\_hybrid`/shipped `gemini`, never `fusion\_hybrid`; and even inside FusionSegmenter the court code only runs when `cue\_flags.person` is True (`fusion\_hybrid.py:88`). The golden eval passes `court\_polygon=None` explicitly (`fusion\_golden.py:108-109`).
- no_special_casing: yes. The court-derived features (`players\_in\_court`, `in\_court\_ratio`, `two\_sided\_motion`) enter the decision only as columns the learned LogisticRegression weights ? `fusion\_compare.py:47-48,59-61` makes the fusion variant differ from shuttle-only solely by `feature\_names`. The only if/else on a court feature is the optional served Gate B (`fusion\_hybrid.py:148`), a config threshold defaulting OFF.
- retained: yes. After the n=6 result (+person ?F1 ?0.021, CI [?0.165,+0.162], CI clears 0: No ? `QUALITY\_ITERATIONS.md:51`), git history on these files shows only feature-add commits, no deletion/disable. The doc keeps it default-OFF and explicitly blames the miss on "no court polygons ? players_in_court counts spectators" (:55), with "Negative/uncertain results are kept on purpose."

Catalyst answer to the KEY QUESTION: yes ? the math activates the moment a polygon is supplied. `fusion\_hybrid.py:126-131` threads `fcfg.get("court\_polygon")` straight into `compute\_person\_features`, so setting `indexing.fusion.court\_polygon` (e.g. when Roora labels arrive) engages the masked path with zero code change. Tests prove the behavioral flip (`test\_fusion\_features.py:54-67`, `:94-101`): counts/ratios/two-sided differ between `LEFT\_HALF`

and `None`. Today it degrades to "count everyone" under `None`.

Main concern: the polygon path's real-video activation is unverified (owner-side, GPU-gated; `fusion\_hybrid.py:108` says the wiring is mock-tested only), and `FusionConfig` does not validate that supplied polygon vertices are normalised 0-1 ? a future pixel-space Roora polygon would mis-mask silently rather than erroring.